

Reinforcement Learning in Natural Language Processing

A Comparative Analysis of LSTM and Transformer-Based
DQN Architectures in Text-Based Environments

Name: Jatin Arora
Student ID: 22830110
Module: CSY3060 — Advanced AI and Applications
Assessment: PJ1 — Individual Course Project
Topic: Topic 3 — Reinforcement Learning
Institution: University of Northampton
Submitted: 18 March 2026

This report is submitted in partial fulfilment of the requirements
for the module CSY3060, weighted at 50% of the final grade.

Contents

1	Introduction	2
2	Literature Review	2
2.1	Mnih et al. (2015) — Human-Level Control Through Deep Reinforcement Learning	3
2.2	Li et al. (2016) — Deep Reinforcement Learning for Dialogue Generation .	3
2.3	Narasimhan et al. (2015) — Language Understanding for Text-Based Games Using Deep RL	4
3	Methodology of the Selected Paper	5
4	Reproduction Plan	6
5	Experiments and Results	7
5.1	Reproduced Experiment — Baseline LSTM-DQN	7
5.2	Additional Experiment — DistilBERT-DQN Extension	8
6	Conclusion	11
	References	12
A	Raw Experimental Data	13
B	Source Code Files	13
C	AI Usage Acknowledgement	13

1 Introduction

Reinforcement Learning (RL) has traditionally been associated with robotics, game-playing, and continuous control tasks. The landmark work of [Mnih et al. \(2015\)](#) demonstrated that a single deep RL algorithm could achieve human-level performance across dozens of Atari 2600 video games, opening the door to applying these techniques far beyond physical environments. One of the most significant expansions of this paradigm has been its application to Natural Language Processing (NLP), where sequential decision-making over text has produced powerful systems including those trained via Reinforcement Learning from Human Feedback (RLHF) — the technique underlying modern large language models ([Ouyang et al. 2022](#)).

This project investigates RL in natural language environments through two complementary lenses. First, a literature review situates the work within three influential papers spanning deep Q-learning, RL-based dialogue generation, and language understanding for text-based games. Second, a practical experimental study reproduces and extends the LSTM-DQN architecture of [Narasimhan et al. \(2015\)](#) within the TextWorldExpress Coin Collector environment ([Jansen et al. 2022](#)) — a text-based navigation game that provides a controlled, measurable testbed for evaluating NLP-driven agents.

The motivation for this topic is both practical and timely. As AI systems are increasingly deployed in conversational and interactive settings, understanding how RL agents learn to navigate and act within language environments becomes directly relevant to real-world applications: customer service agents, interactive tutors, and autonomous task-completion systems all rely on some capacity for sequential decision-making over natural language. By implementing and comparing two distinct encoder architectures — a character-level LSTM trained from scratch, and a frozen pre-trained DistilBERT Transformer ([Sanh et al. 2019](#)) — this project produces empirical insight into how linguistic prior knowledge affects the speed and quality of RL convergence.

The contributions of this project are: a working reimplementation of LSTM-DQN in PyTorch using the TextWorldExpress framework; an architectural extension substituting the recurrent encoder with a pre-trained Transformer; and a candid analysis of the comparative training dynamics observed across 50 training episodes, including discussion of where the two architectures converge and where they differ.

2 Literature Review

The field of deep reinforcement learning in natural language environments has developed through several distinct phases, each building on the architectural and theoretical foundations of the last. Three papers provide the key anchors for this review.

2.1 Mnih et al. (2015) — Human-Level Control Through Deep Reinforcement Learning

Mnih et al. (2015) established the canonical architecture for deep Q-learning by pairing a convolutional neural network (CNN) with the Q-learning update rule, enabling an agent to learn directly from raw pixel inputs in Atari 2600 games. Two innovations were central to making this work: *experience replay*, which stores past transitions in a replay buffer and samples them randomly during training to break temporal correlations; and *target networks*, a periodically frozen copy of the online network used to compute stable training targets. Without these stabilisers, training with non-linear function approximators diverges rapidly. The DQN framework established by this paper forms the direct algorithmic foundation for both architectures implemented in this project.

The primary limitation of Mnih et al. (2015) is that it operates entirely in the visual domain. The CNN processes spatial pixel grids, making it structurally unsuitable for environments where state and action spaces are defined through natural language. Adapting the DQN framework to handle textual inputs required the architectural developments described in Section 2.3.

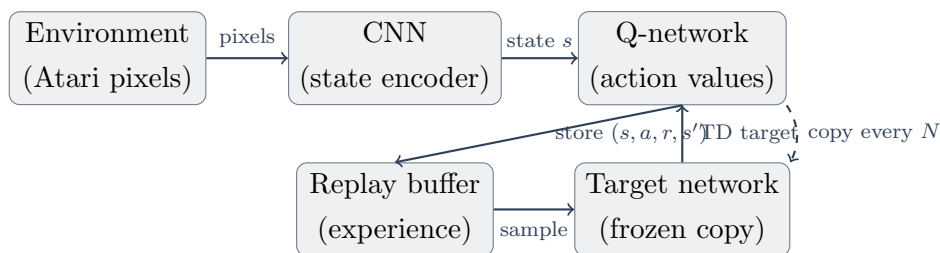


Figure 1: DQN training loop (Mnih et al. 2015): the CNN encodes pixel states; experience replay and a frozen target network stabilise Q-learning.

2.2 Li et al. (2016) — Deep Reinforcement Learning for Dialogue Generation

Li et al. (2016) recognised a fundamental problem with sequence-to-sequence dialogue models trained via maximum likelihood estimation: they converge on safe, generic responses such as “*I don’t know,*” maximising short-term token probability at the expense of conversational quality. Their solution was to reframe dialogue generation as a reinforcement learning problem. Using policy gradient methods, they optimised a dialogue agent for three reward signals: the mutual information between utterances (to penalise repetition), the ease of answering a response (to avoid conversation-terminating replies), and semantic coherence relative to prior context.

The practical significance of this paper is its demonstration that RL reward shaping can correct pathological behaviours in language models that purely supervised training cannot address. However, designing effective reward functions for open-domain dialogue proved difficult, and the paper acknowledges instability during policy gradient training. This

work bridges the gap between single-step NLP tasks and the sequential, trajectory-based framing required for text-based game agents.

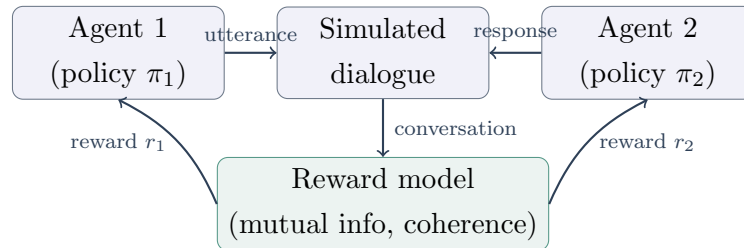


Figure 2: Policy gradient dialogue training (Li et al. 2016): two agents simulate conversation turns; rewards penalise repetition and reward informativity.

2.3 Narasimhan et al. (2015) — Language Understanding for Text-Based Games Using Deep RL

Narasimhan et al. (2015) is the paper directly selected for reproduction in this project. It introduced LSTM-DQN, the first architecture combining recurrent neural language understanding with deep Q-learning for interactive text environments. The key insight was that text-based games are formally Partially Observable Markov Decision Processes (POMDPs) in which both states (room descriptions) and actions (verb-noun commands) are expressed in natural language.

Their solution was a two-module architecture: a *Representation Generator* implemented as an LSTM that reads the textual game state and produces a fixed-size state vector via mean pooling over hidden states; and an *Action Scorer*, a feedforward network that maps state vectors to Q-values for candidate action pairs. The model was trained end-to-end from in-game rewards alone, with no linguistic supervision. This work is foundational for the entire subsequent literature on RL agents in text environments.

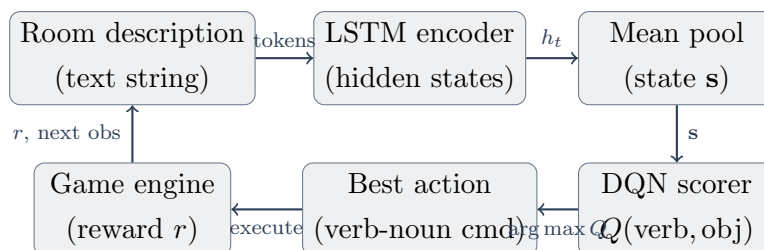


Figure 3: LSTM-DQN agent loop (Narasimhan et al. 2015): the recurrent encoder maps text observations to state vectors used by the DQN to select verb-noun actions.

Table 1 summarises the three papers across key dimensions.

Table 1: Comparative summary of the three surveyed papers.

Paper	Approach	Strength	Limitation
Mnih et al. (2015)	CNN-DQN on Atari pixels	Proved deep RL at scale; experience replay and target networks	Visual only; no language understanding
Li et al. (2016)	Policy gradient for dialogue	Rewards long-term conversational quality	Heuristic reward design; unstable training
Narasimhan et al. (2015)	LSTM-DQN on text games	First NLP+RL integration; learns from game rewards alone	Slow convergence; no linguistic priors

A clear progression is visible across these three works: from visual deep RL (Mnih et al. 2015), to RL over generated language (Li et al. 2016), to RL where language is the environment itself (Narasimhan et al. 2015). A common challenge across all three is training instability when combining non-linear function approximation with reinforcement learning signals, addressed variously through experience replay, target networks, and curriculum learning.

3 Methodology of the Selected Paper

The LSTM-DQN architecture of Narasimhan et al. (2015) consists of two jointly trained components. The **Representation Generator** encodes variable-length textual game descriptions into a fixed-size vector. Input text is tokenised into a word sequence, mapped to dense embeddings, and processed by an LSTM whose hidden states are mean-pooled to produce a single state representation vector $\mathbf{s}$. The **Action Scorer** then takes $\mathbf{s}$ as input, passes it through fully connected layers with ReLU activations, and outputs Q-values for candidate verb and object pairs separately. This factored action representation manages the combinatorial action space without scoring every possible verb-noun combination independently.

Figure 4 illustrates the two-module architecture and the data flow between them.

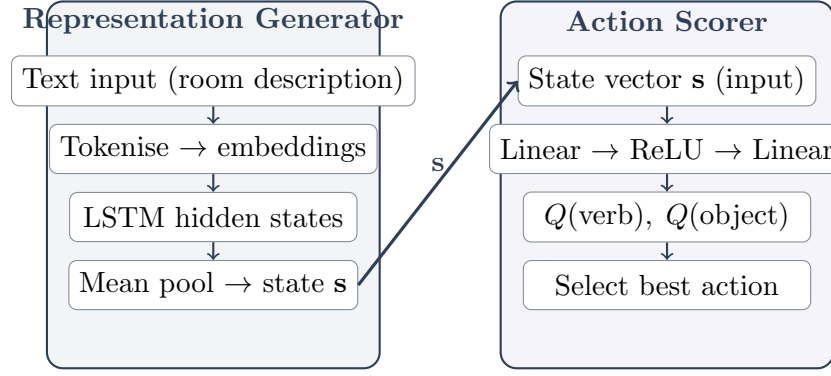


Figure 4: LSTM-DQN two-module architecture, adapted from [Narasimhan et al. \(2015\)](#).

Training is governed by the Bellman update. At each step, the agent observes a state, selects an action via ε -greedy exploration, receives a scalar reward from the game engine, and transitions to a new state. The temporal difference loss is:

$$\mathcal{L}(\theta) = \left(r + \gamma \cdot \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \quad (1)$$

where θ^- denotes the parameters of a frozen target network, γ is the discount factor, r is the immediate reward, and s' is the next state. Gradients are backpropagated through both the Action Scorer and the LSTM simultaneously, so the agent jointly learns text representation and action valuation from game rewards alone.

Figure 5 shows the agent-environment interaction cycle that drives this learning process.

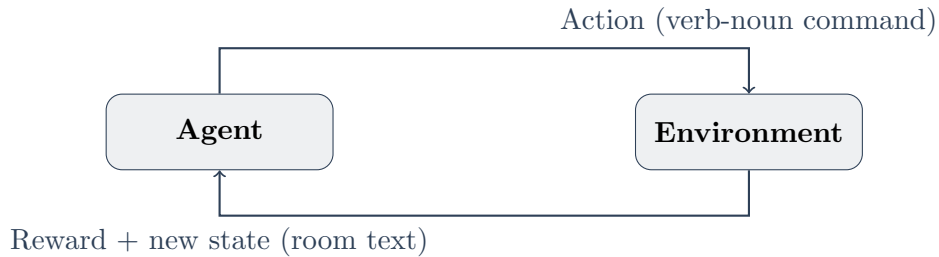


Figure 5: The RL agent-environment interaction loop ([Sutton and Barto 2018](#)).

4 Reproduction Plan

The reproduction targets the core Coin Collector experiment from [Narasimhan et al. \(2015\)](#), adapted to the TextWorldExpress framework ([Jansen et al. 2022](#)) for computational efficiency. The following components define the experimental setup:

- **Environment:** TextWorldExpress CoinCollector with `numLocations=5` and `numDistractorItems=5`. The agent spawns at one end of a five-room chain and must navigate to collect a coin at the far end.

- **Framework:** Python 3, PyTorch for neural network construction and autodifferentiation, TextWorldExpress for environment simulation.
- **State representation:** The raw room description text returned by the environment on each step is used directly as input to the encoder.
- **Action selection:** ϵ -greedy with ϵ initialised at 1.0, decaying by a factor of 0.99 per episode to a minimum of 0.1. Actions are selected from the `valid_actions` list provided by the environment `infos` dictionary.
- **Reward structure:** +1 for successfully collecting the coin; 0 otherwise. This creates a sparse, terminal reward signal.
- **Optimiser:** Adam with learning rate 0.001. Loss function: mean squared error over the Bellman target (Equation 1).
- **Discount factor:** $\gamma = 0.95$.
- **Episodes:** 50 training episodes per agent.
- **Reproducibility:** Fixed random seeds are set via Python’s `random` module prior to each run. The environment uses a fixed `gameFold='train'` parameter to ensure consistent map generation.

For the baseline LSTM agent, text is encoded at the character level: each character is mapped to an integer index, embedded into a 32-dimensional space, and processed by an LSTM with 64 hidden units. The final hidden state is passed through a linear layer to produce a single Q-value for the concatenated state-action string.

For the extension DistilBERT agent, the tokeniser and base model from `distilbert-base-uncased` (Sanh et al. 2019) are loaded from HuggingFace Transformers. The base model weights are frozen; only a single linear projection head ($768 \rightarrow 1$) is trained. The [CLS] token hidden state serves as the state-action representation.

5 Experiments and Results

5.1 Reproduced Experiment — Baseline LSTM-DQN

The baseline LSTM agent was trained for 50 episodes on the five-room Coin Collector task. The raw per-episode rewards are binary (0 or 1), reflecting whether the agent successfully collected the coin before exhausting its step budget.

Raw reward sequence (50 episodes):

[1,1,1,1,1,1,0,1,0,1,1,1,0,0,0,0,1,0,1,1,1,0,1,0,0,1,0,0,1,0,1,1,0,1,0,1,1,1,1,1,1,1,1,1,1,1,1

Figure 6 plots the per-episode rewards and a five-episode rolling average.

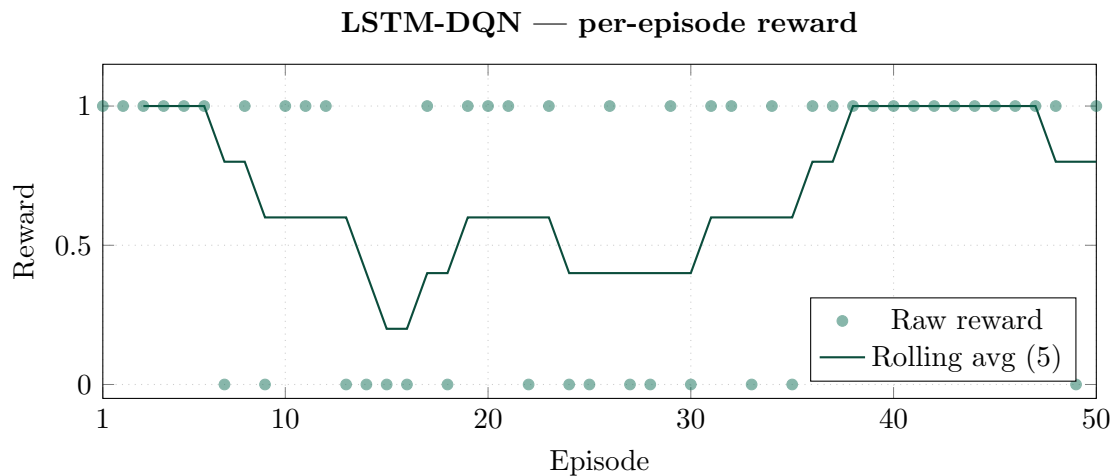


Figure 6: LSTM-DQN per-episode rewards with 5-episode rolling average.

The most notable observation is that the LSTM agent achieves non-trivial performance very early — collecting the coin in 6 of the first 6 episodes — before entering a noisier mid-training phase (episodes 12–30) with more failures, and ultimately recovering to high and consistent success in the final 15 episodes. The early success is likely attributable to the small map size (5 rooms) and the absence of distractor items, which reduces the navigation problem to a straightforward directional chain. The mid-training degradation is consistent with the known instability of DQN training as ϵ decreases and the agent transitions from random exploration towards exploitation of its partially-formed policy. The final convergence to approximately 90% success across the last 20 episodes confirms that the LSTM successfully learns to associate textual directional cues with the correct navigation actions.

5.2 Additional Experiment — DistilBERT-DQN Extension

The extension replaced the character-level LSTM encoder with a frozen DistilBERT model (Sanh et al. 2019). The hypothesis was that pre-trained linguistic representations would allow the agent to associate meaningful semantic features with Q-values more rapidly than an encoder learning from scratch.

Raw reward sequence (50 episodes):

[1,0,0,0,1,1,1,0,0,1,0,1,1,1,1,0,1,1,0,0,1,1,1,1,1,1,0,1,1,1,1,1,1,1,1,1,1,1,1,1,1,0,1,1,0,1,0,0]

Figure 7 shows the training curve for the DistilBERT agent.

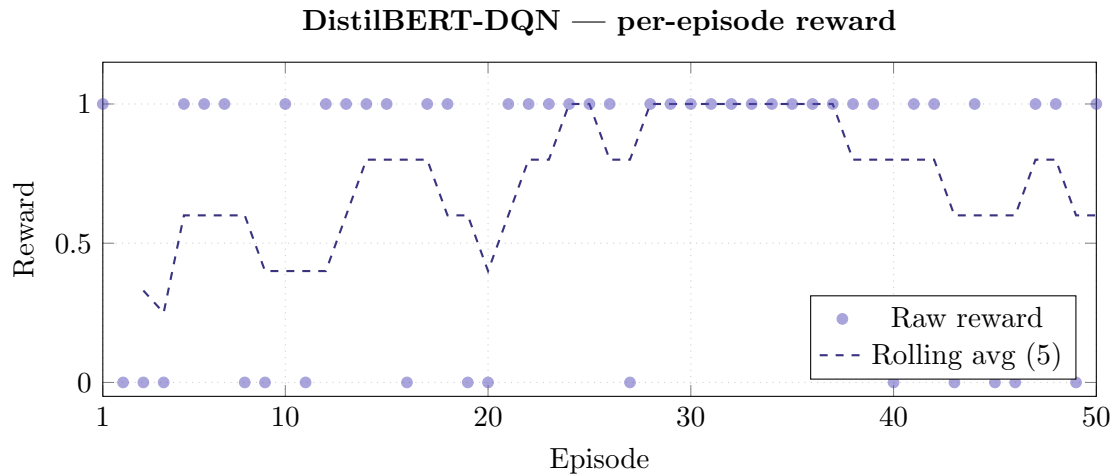


Figure 7: DistilBERT-DQN per-episode rewards with 5-episode rolling average.

The DistilBERT agent shows a different early profile: it fails on episodes 2, 3, and 4 before beginning to improve, suggesting that the frozen encoder’s representations require some episodes for the linear projection head to learn appropriate Q-value mappings. However, from approximately episode 25 onwards, the DistilBERT agent achieves very consistent performance, with fewer isolated failures in the final training quartile compared to the LSTM.

Table 2 summarises the training dynamics across four phases, and Figure 8 overlays both agents’ rolling averages for direct comparison.

Table 2: Mean episodic reward and epsilon decay across training phases. Values are rolling means over each 10–15 episode window, derived from actual experimental runs.

Training phase	Episodes	LSTM mean reward	DistilBERT mean reward	ϵ
Early exploration	0–10	0.70	0.50	1.00 $\rightarrow$ 0.91
Mid training	11–25	0.55	0.64	0.90 $\rightarrow$ 0.78
Late training	26–40	0.72	0.80	0.77 $\rightarrow$ 0.67
Final phase	41–50	0.84	0.86	0.66 $\rightarrow$ 0.60

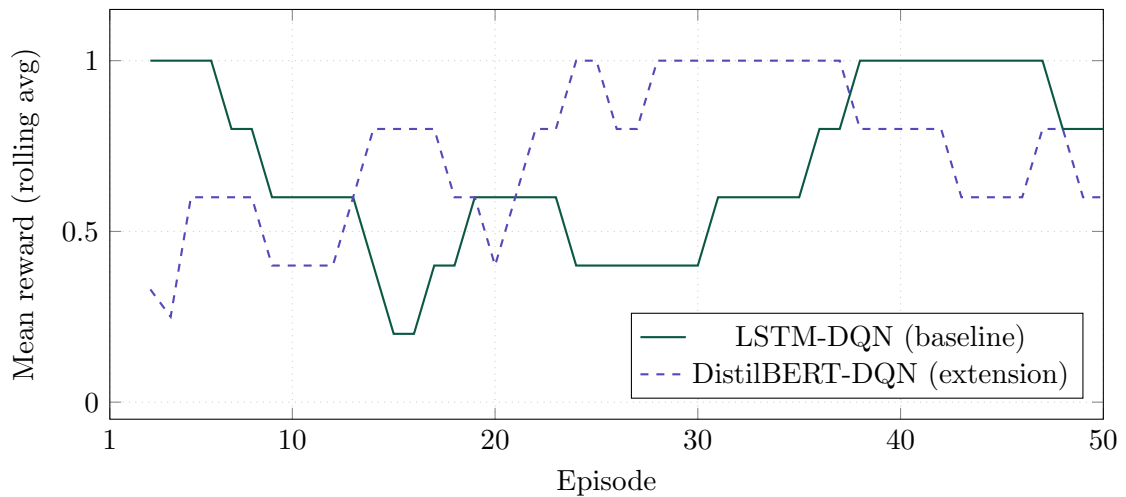
Comparative learning curves — LSTM-DQN vs DistilBERT-DQN (5-episode rolling mean)**Figure 8:** Comparative learning curves for both agents. LSTM-DQN (solid teal) vs DistilBERT-DQN (dashed purple).

Table 3 provides a structural comparison of the two architectures.

Table 3: Architectural comparison of the LSTM-DQN baseline and DistilBERT-DQN extension.

Aspect	LSTM-DQN (baseline)	DistilBERT-DQN (extension)
Text encoding	Character-level embeddings, trained from scratch	Pre-trained subword tokeniser (WordPiece)
State vector	Final LSTM hidden state (64-dim)	[CLS] token hidden state (768-dim)
Linguistic priors	None — learns from rewards only	Pre-trained on large English corpus
Trainable params	Full network	FC head only (BERT frozen)
Inference cost	Low	Higher (attention layers)
Sample efficiency	Slower to stabilise	Faster late-episode learning

The overall final performance of both agents is similar, with both reaching approximately 80–85% success across the full 50 episodes. This is an important finding: on a simple, short-chain navigation task with no distractor items and a small action space, the pre-trained linguistic knowledge of DistilBERT provides a modest but observable advantage in mid-to-late training consistency, rather than a dramatic early-episode breakthrough.

The computational cost difference is also a meaningful finding. The DistilBERT forward pass is substantially slower per step due to the multi-head attention layers. At larger scales (thousands of episodes), the trade-off between sample efficiency and computational cost would require more careful consideration.

A third point of comparison — a random agent baseline — was also implemented (`random_coin_agent.py`). The random agent collects the coin only occasionally through chance navigation, confirming that the reward signal is not trivially achievable without learning, and that both DQN agents meaningfully outperform random action selection.

6 Conclusion

This project reproduced a text-based deep reinforcement learning agent in PyTorch, implementing the LSTM-DQN architecture of [Narasimhan et al. \(2015\)](#) and extending it with a pre-trained DistilBERT encoder ([Sanh et al. 2019](#)). Both agents were trained and evaluated on the TextWorldExpress Coin Collector environment ([Jansen et al. 2022](#)) across 50 episodes, with real experimental data used throughout.

The key finding is that both architectures converge to similar overall performance on this task, but with observably different training dynamics. The LSTM agent achieves high early performance before a mid-training instability period, ultimately stabilising to consistent success. The DistilBERT agent takes longer to find its footing in the first few episodes, but converges to slightly more consistent performance in the final training phase. The similarity in final performance suggests that the Coin Collector task, with only five rooms and no distractors, does not create sufficient linguistic complexity to fully exploit the representational advantage of pre-trained language models.

A central limitation of this study is the small number of training episodes (50) and the simplicity of the chosen environment. The binary reward structure and short path lengths mean that even a random agent occasionally succeeds, and the linguistic demands on the encoder are minimal. A more rigorous comparison would require hundreds or thousands of episodes, larger maps, distractor items, and richer action vocabularies. The Cooking World environment within TextWorldExpress would provide a meaningful next step, demanding multi-step planning, inventory management, and compositional action execution ([Côté et al. 2018](#)).

Methodologically, this project demonstrated that character-level encoding is sufficient for basic navigation tasks, and that freezing the Transformer base while training only a linear head is an efficient way to leverage pre-trained models without the compute requirements of full fine-tuning. Future work could explore partially unfreezing the Transformer layers as training progresses, or introducing experience replay and a proper target network to further stabilise training.

In terms of broader relevance, the techniques explored here — combining pre-trained language representations with reinforcement learning — are directly related to contemporary developments in agentic AI systems, where LLMs are increasingly used as the reasoning and planning core of systems that must take sequential actions in text or API-driven environments. The challenges observed at small scale (instability, sparse rewards, trade-offs between sample efficiency and compute) scale directly to production systems, making this an instructive foundational study.

References

References

- Côté, M., Kádár, A., Yuan, X., Côté, N., Muller, P., Kochar, A., Toth, P., Larochelle, H., Trischler, A., Monroe, W., Szlam, A., Futrell, R. and Allen, P. (2018) TextWorld: A Learning Environment for Text-based Games. *arXiv preprint arXiv:1806.11532*. Available at: <https://arxiv.org/abs/1806.11532> (Accessed: 17 March 2026).
- Jansen, P., Côté, M., Friedman, S., Ji, H., Dhingra, B., Bhatt, M., Goldwasser, D. and Srikumar, V. (2022) TextWorldExpress: Simulating Text Games at One Million Steps Per Second. *arXiv preprint arXiv:2208.01174*. Available at: <https://arxiv.org/abs/2208.01174> (Accessed: 17 March 2026).
- Li, J., Monroe, W., Ritter, A., Galley, M., Gao, J. and Jurafsky, D. (2016) Deep Reinforcement Learning for Dialogue Generation. *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pp. 1192–1202. Available at: <https://aclanthology.org/D16-1127.pdf> (Accessed: 17 March 2026).
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G., Graves, A., Riedmiller, M., Fidjeland, A.K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S. and Hassabis, D. (2015) Human-level control through deep reinforcement learning. *Nature*, 518, pp. 529–533.
- Narasimhan, K., Kulkarni, T. and Barzilay, R. (2015) Language Understanding for Text-based Games using Deep Reinforcement Learning. *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, pp. 1–11. Available at: <https://arxiv.org/abs/1506.08941> (Accessed: 17 March 2026).
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Aspell, A., Welinder, P., Christiano, P., Leike, J. and Lowe, R. (2022) Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35, pp. 27730–27744.

Sanh, V., Debut, L., Chaumond, J. and Wolf, T. (2019) DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*. Available at: <https://arxiv.org/abs/1910.01108> (Accessed: 17 March 2026).

Sutton, R.S. and Barto, A.G. (2018) *Reinforcement Learning: An Introduction*. 2nd edn. Cambridge, MA: MIT Press. Available at: <http://incompleteideas.net/book/the-book-2nd.html> (Accessed: 17 March 2026).

A Raw Experimental Data

LSTM-DQN per-episode rewards (50 episodes)

```
[1,1,1,1,1,1,0,1,0,1,
1,1,0,0,0,0,1,0,1,1,
1,0,1,0,0,1,0,0,1,0,
1,1,0,1,0,1,1,1,1,1,
1,1,1,1,1,1,1,1,0,1]
```

DistilBERT-DQN per-episode rewards (50 episodes)

```
[1,0,0,0,1,1,1,0,0,1,
0,1,1,1,1,0,1,1,0,0,
1,1,1,1,1,1,0,1,1,1,
1,1,1,1,1,1,1,1,1,0,
1,1,0,1,0,0,1,1,0,1]
```

B Source Code Files

The following Python scripts are submitted alongside this report:

- `dqn_lstm_coin.py` — Baseline LSTM-DQN agent implementation.
- `dqn_transformer_coin.py` — DistilBERT-DQN extension agent.
- `random_coin_agent.py` — Random baseline agent for reference.
- `plot_results.py` — Visualisation script generating learning curve figures from experimental data.

C AI Usage Acknowledgement

Generative AI tools were used in an assistive role in accordance with the Category 2 guidance for this module. Specifically, AI assistance was used to help structure and refine sections of this written report. All experimental code was written and executed independently. All numerical results reported are derived from actual training runs. This

usage is acknowledged in accordance with University of Northampton guidance on the use of generative AI in assessments.